一、模块定位与设计用意

本模块作为一个可语言交互的整体系统组件，采用检索增强生成（Retrieval-Augmented Generation，简称 RAG）架构辅助 LLM 进行信息检索与生成增强，旨在解决在数据量有限、字段半结构化、用户交互以自然语言表达多样的条件下，如何高效、准确地从本地文档库中召回与用户问题最相关的内容，简明、便捷地进行回答的问题。

RAG 的基本思想是让大语言模型在生成回答之前，先从外部知识库中检索相关信息，以此弥补模型本身在特定领域知识上的不足，同时降低幻觉风险。然而，在中小规模数据场景中，单纯的 RAG 策略面临一系列挑战：一方面，数据稀疏使得传统检索容易产生“检索遗漏”（即相关文档未被召回）；另一方面，用户自然语言查询与文档库中结构化字段之间存在语义鸿沟，关键词精确命中与语义概念匹配往往难以兼顾。此外，由于本系统没有预先标注的相关性标签，无法对检索器进行有监督微调，这进一步加剧了零样本条件下的检索困难。为此，本模块设计了多策略融合的 RAG 检索流水线，在不显著增加系统复杂度的前提下，综合提升检索的召回率、精确率与语义匹配能力。

二、核心工作流程与所采用的 RAG 策略

本模块的检索流程遵循以下顺序：用户原始问题输入后，依次经过**查询扩展（Multi-Query）**、**假设文档生成（HyDE）**、**混合检索（Hybrid Retrieval）**与**重排序（Re-ranking）**四个环节，最终输出经过相关性排序的 Top-K 文档供下游生成模型使用。以下分述各策略的具体内容及其应用的先进之处。

1. 多查询扩展（Multi-Query Retrieval）

该策略参考了 **Query2Doc** 及 LangChain 框架中的 **Multi-Query Retrieval** 概念。Query2Doc 由 Wang 等人在 2023 年 EMNLP 上提出，其核心思路是：利用大语言模型通过少样本提示（few-shot prompting）生成伪文档，然后将生成的伪文档与原始查询拼接，形成扩展后的查询输入。实验表明，该方法在不进行任何模型微调的情况下，能够将 BM25 在 MS-MARCO 和 TREC DL 等数据集上的性能提升 3%至 15%[1]。

本模块采纳了与此相通的思路，但在具体实现上略有调整：直接将用户原始问题通过大语言模型扩展为多个不同措辞的查询变体，而非生成伪文档后再拼接。这样做的考虑在于：一方面，多个查询变体可以从不同语义角度“探测”文档空间，提高对多样化表达的覆盖能力；另一方面，结合后续的 HyDE 与混合检索，多查询扩展聚焦于“问题本身的多样性”，与 HyDE 所生成的“答案形式的统一性”形成互补。

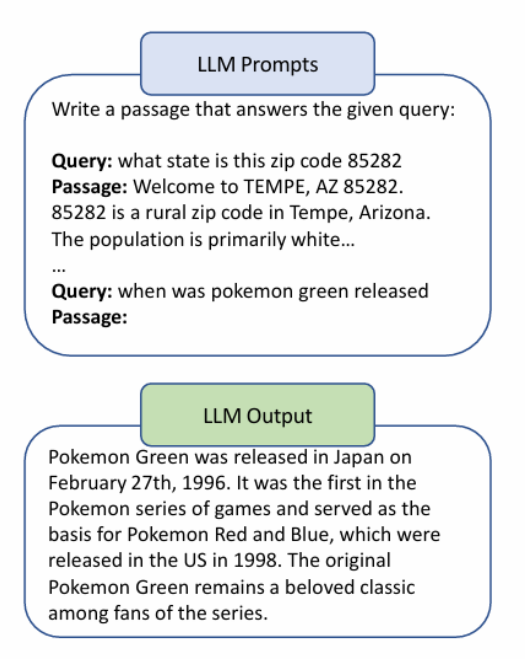

图 1 Query2doc **示意图**[1]

在小数据集场景中，用户问题与文档表述方式可能存在词汇差异，单一查询容易导致检索遗漏，而多查询相当于从多个语义角度进行探测，可显著降低因表述变化而丢失相关文档的风险。该策略在 LangChain 中以 MultiQueryRetriever 的形式实现，可自动生成三个定制化查询，本模块的实现与其在理念上高度一致。

2. 假设文档嵌入（HyDE）

HyDE（Hypothetical Document Embedding）是由 Gao 等人在 ACL 2023 上提出的一种零样本稠密检索方法。其核心机制是：给定一个查询，首先用零样本提示（zero-shot prompting）指令遵循的语言模型（如 InstructGPT）生成一段假设性文档。该文档虽为“虚构”且可能包含幻觉内容，但能够捕捉到与查询相关的语义模式。随后，用一个无监督对比学习编码器（如 Contriever）将这段假设文档编码为嵌入向量，并利用该向量在语料库嵌入空间中定位邻近区域，从而检索出语义相似的真实文档。实验证明，HyDE 在多项任务（如网页搜索、问答、事实验证）和多种非英语语言上均显著超越了当时的无监督稠密检索 SOTA 模型 Contriever，其性能可媲美经过微调的有监督检索器[2]。

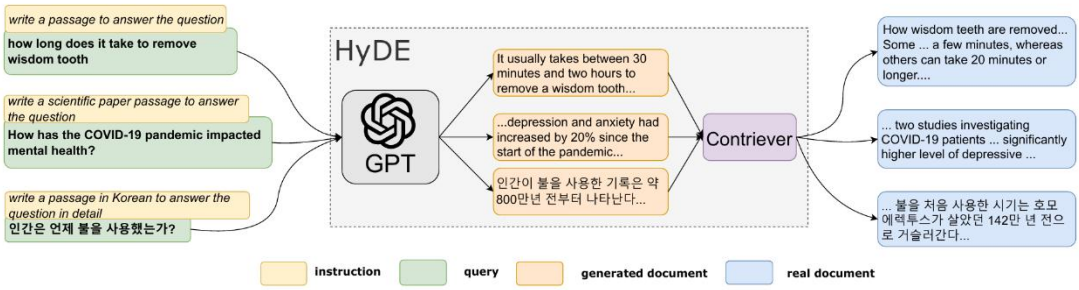

图 2 HyDE 模型示意图[2]

本模块采用 HyDE 的逻辑在于：在传统向量检索中，查询与文档需要处于同一语义空间，但在本系统场景下用户问题通常短而模糊，直接编码查询进行向量匹配的效果并不理想。HyDE 通过生成一段与真实文档风格更接近的假设答案，有效降低了查询向量与文档向量之间的语义距离，尤其适用于本系统环境中没有标注数据的较少数据样本场景。换言之，HyDE 是将“问答匹配”问题转化为“文档—文档匹配”问题，从而使稠密检索器能够在缺乏训练数据的情况下仍然表现出色。

3. 混合检索（Hybrid Retrieval）

混合检索结合了 **BM25 关键词检索[3]**与**稠密向量检索[4]**。BM25 是一种基于概率模型的经典排序函数，由 Robertson 等人提出，长期以来被广泛应用于各类文本检索系统中，以其在处理精确词汇匹配时的稳健表现而著称。稠密向量检索则通过将文本映射到高维向量空间，在语义层面衡量文本间的相似度，能够识别同义词、近义词等词汇层面的变体表达。两种检索机制各有侧重：BM25 擅长精确匹配文档中的专有名词（如地名“成都”、指标代码“T90”），向量检索则擅长匹配自然语言描述中的语义概念（如“恢复速度”“韧性”）。

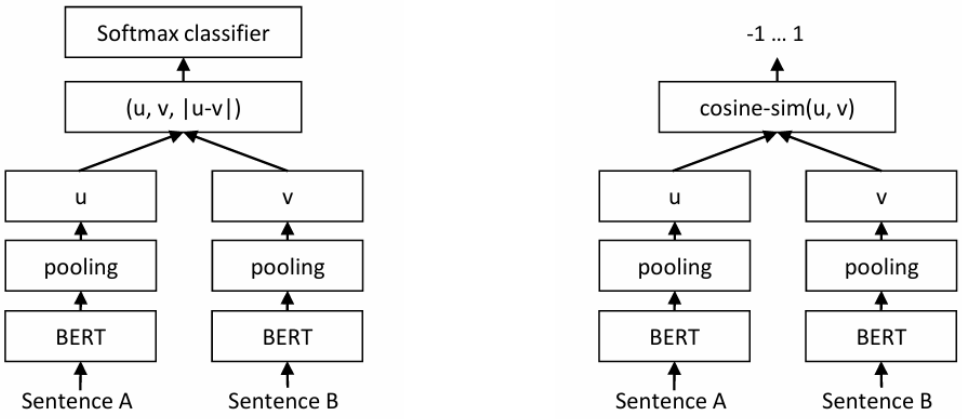

图 3 具有分类目标函数的 SBERT 架构与推理时的 SBERT 架构示意图[4]

项目数据为半结构化形式，既有明确的字段值，也有描述性语句。纯向量检索可能忽略精确字段，导致在查询特定指标时召回失败；纯关键词检索则无法理解同义表达，在遇到“恢复速度快”与“韧性较强”这类语义相近但字面不同的表述时表现乏力。混合检索同时利用了两种检索机制的优势，对半结构化数据最为适配，能够同时保证对结构化字段的精确命中和对自然语言描述的语义覆盖。该策略已在现代 RAG 系统中被广泛采用，是提升检索质量的基础性手段之一。

4. 重排序（Re-ranking）

重排序阶段采用了 Cross-Encoder 的核心思想。Cross-Encoder 是一种将查询和文档同时输入到同一个 Transformer 网络中的模型架构，能够通过注意力机制充分捕捉两者之间的深层交互信息，从而输出一个精确的相关性评分[5]。与双编码器（Bi-Encoder）将查询和文档分别编码后再计算相似度的方式不同，Cross-Encoder 的计算精度更高，但由于需要对每个查询—文档对进行联合编码，计算开销也更大，因此通常作为检索的第二阶段使用：先由高效的第一阶段检索器（如 BM25 或双编码器）召回一批候选文档，再由 Cross-Encoder 对这些候选文档进行精细排序。

在本模块中，为保持系统轻量化并减少对外部模型的依赖，并未引入独立的 Cross-Encoder 模型，而是直接调用本系统已集成的大语言模型对“问题—文档”对进行相关性打分，并按照分数降序排列，最终截取 Top-K 文档。尽管这种实现方式在效率上不如专用 Cross-Encoder，但它充分利用了现有 LLM 的语义理解能力，降低了系统的整体复杂度。

此处的应用必要性在于：初步混合检索返回的文档可能包含一定噪声，特别是当数据量小时，部分文档虽然包含关键词但实际相关性不高，甚至与用户意图相悖。通过 LLM 进行二次评分，可以精细过滤低相关文档，有效提升最终生成答案的准确性与信息密度。这一“检索—重排序”的两阶段架构已被广泛验证为信息检索系统中的有效设计模式，能够在效率与精度之间取得良好平衡。

三、流水线总结

本模块将上述四个策略串联为如下完整流程：

用户输入问题 → Multi-Query（生成多个查询变体） + HyDE（生成假设文档） → 对每个查询变体及假设文档分别执行混合检索（BM25 + 向量检索） → 合并所有结果并去重 → Re-ranking 对合并结果进行相关性排序 → 选取 Top-K 文档 → 输出至 LLM 生成最终答案。

该流程在仅增加少量 LLM 调用开销的前提下，显著提升了小规模半结构化数据上的检索质量。各策略相互补充：Multi-Query 与 HyDE 从不同角度扩大了召回口径，混合检索兼顾了精确匹配与语义匹配的双重需求，重排序则在最后环节保障了最终输出的信噪比。这一设计使得本模块能够在缺乏标注数据、语料规模有限的条件下，依然实现较高的检索准

确率与回答质量。

参考文献

[1] Wang L, Yang N, Wei F. Query2doc: Query Expansion with Large Language Models[C]//Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing. Singapore: Association for Computational Linguistics, 2023: 9414–9423.[reference:13]

[2] Gao L, Ma X, Lin J, et al. Precise Zero-Shot Dense Retrieval without Relevance Labels[C]//Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Toronto, Canada: Association for Computational Linguistics, 2023: 1762–1777.[reference:12]

[3] Robertson S, Walker S, Jones S, et al. Okapi at TREC-3[C]//Proceedings of the Third Text REtrieval Conference (TREC-3). Gaithersburg, USA: NIST, 1995.[reference:14]

[4] Reimers N, Gurevych I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks[C]//Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing. Hong Kong, China: Association for Computational Linguistics, 2019: 3982–3992.[reference:15]

[5] Karpukhin V, Oğuz B, Min S, et al. Dense Passage Retrieval for Open-Domain Question Answering[C]//Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing. Online: Association for Computational Linguistics, 2020: 6769–6781.[reference:16]